

Efficient Vehicle Plate Recognition using LPRNET

(srames25 - Sri Roopa Ramesh Babu, cmallu - Chandana Mallu, srajara4 - Sai Krishna Rajaraman)

GitHub Folder Link : [LPRNet repository](#)

Google Colab Link : [Colab Notebook](#)

Original Model - Components, Methods and Set Up.

The original DNN model utilized in the project is [LPRNet \(License Plate Recognition Network\)](#) which is an end-to-end pre-trained CNN model that detects vehicle plates without requiring any pre-processing and RNNs for text detection. The network consists of multiple components that articulate into the end-to-end model.

Structural Components

1. **Backbone Network** : The backbone network consists of convolutional layers that considers all three channels of the input image for extracting low level features that can be leveraged for analysis. It also contains Batch Normalization and ReLU activation in order to normalize the sample space and apply non-linearity to the extracted features. A 3 dimensional Max Pooling layer also exists within the backbone network effectively down sampling the spatial features. A distinctive component in the model is the small_basic_block which works towards reducing computational complexity while retaining the required spatial features. Dropout layers have been introduced in order to prevent overfitting by regularizing the intermediate features before the final classification is done.
2. **Small Basic Block** : The small_basic_blocks introduced in the backbone network are reusable convolutional operations with a ReLU activation that reduces the dimensionality, improves relevant feature extraction in both horizontal and vertical directions while also integrating and preserving the dimensionality. This component plays an important role in the spatial feature extraction part of the backbone network while reducing the overall computational complexity of the network.
3. **Container Block** : The container block combines the extracted features from the backbone network along with the additional contextual features called the global context vector thus ensuring dimensionality reduction for the classification task.

Functional Components

1. **Global Context** : The model utilizes aggregation and normalization methods to understand the global feature patterns as spatial feature retainance is essential for the

objective task of vehicle plate detection. This functionality is enforced by collecting features from specific convolutional layers from the backbone network thus enabling the capture of low-level, abstract patterns. Normalization is done on the features, in order to ensure that all layers contribute towards the final output by considering mean squared values to balance the individual layer contributions. The final feature maps after pooling and normalization are concatenated to form the global context vector.

2. **Forward Pass** : The forward pass network processes the functionalities for the global context vectors and its corresponding intermediate feature maps by retaining those values during the initial backbone network pass for multi-level feature representation. These maps are pooled to reduce dimensions and normalized to add equal contributions towards the final classification. These processed feature maps are concatenated to form the global context which is passed through the container for further dimensionality reduction. The final output is obtained by averaging the spatial dimensions constituting the logits which represent the predictions of the license plate.

Model Set up

1. Several parameters are used in the model initialization including,
 - `lpr_max_len` : This indicates the maximum length of the characters that can be present in the license plate.
 - `phase` : This indicates if the model is in its “train” phase or “evaluation” phase.
 - `class_num` : This parameter shows the number of possible classes based on the training set(66) which has been updated to (68) post the recent dataset expansion.
 - `dropout_rate` : This parameter indicates the percentage of neurons dropped in order to avoid overfitting during the training phase.
2. The core backbone network consists of parameters pertaining to the computations.
 - `in_channels` : This indicates the number of input channels considered for the model which is 3 in this case (RGB).
 - `out_channels` : This parameter represents the channels in the output generated which is 64 in this model.
 - Various `kernel_size` values have been considered in order to perform average pooling for the global context.
 - Various `stride` values are also considered in order to support the pooling and downsampling functionalities.
3. The `small_basic_block` network contains parameters to adjust the input and output channels which in this case are variant across 64,128 and 128,256 depending on the feature map shape.
4. The output parameter *logit* is obtained as a tensor containing the batch size, class number and width of the image.

Due to focus on the model and MLC optimization aspects, the pre-trained model weights have been used for the optimizations. *No re-training* is done on the original model due to the unavailability of training data.

The training and testing scripts provided for executing and evaluating the model contain custom functions like `collate_fn` and `greedy_decode_eval` for custom data loading and evaluation respectively. The `collate_fn()` is used to load data as batches based on the label length. The `greedy_decode_eval()` function is customized to evaluate the model accuracy by considering the true positives and false negatives and also logs the performance metrics for the particular call.

Model Optimizations

Fusion

Fusion is a model optimization technique often done along with quantization in order to reduce total number of computations resulting in efficient computations. For this implementation, [Conv2D+BatchNorm+ReLU](#) layer combinations have been fused into single layers. Even though it did not result in a compressed file size, the computations required and running time have decreased significantly while still retaining the test accuracy.

This fusion technique impacted the accuracy by 2% but offers a significant reduction in inference time and while also retaining the model size.

Quantization

Quantization is helpful to reduce the number of FLOPS for a model while inherently also reducing memory, and speeding up the inference process. Due to the unavailability of training data, we pursued Post Training Quantization and Dynamic Quantization in order to quantize the weights post the training using the extracted weight folder.

Static Quantization

Post Training Static Quantization:

This technique converts the floating-point weights of a pre-trained neural network into lower-precision integer representations after the training process is complete. By converting 32-bit floating-point values to 8-bit integers, static quantization significantly reduces model size and improves inference speed, particularly on hardware with optimized integer arithmetic operations.

This gave us a test accuracy of 90.00% with the inference time being 12457.27ms which is not good in comparison to our baseline model accuracy and timings.

Dynamic Quantization

Dynamic Quantization differs from static quantization in its approach to handling activations during inference. While static quantization pre-computes quantization parameters, dynamic quantization performs the quantization of activations on-the-fly during model execution. This technique only quantized the weights and biases during the model conversion phase, leaving the activations in floating-point format until runtime. During inference, the activations are temporarily quantized to perform computations and then dequantized back to floating-point format for the next layer. This approach offers greater flexibility in handling varying activation distributions but may incur additional computational overhead compared to static quantization due to the repeated quantization-dequantization operations.

This technique yielded an accuracy of 90.10% and an inference time of 62.75 ms which is comparatively better performing than the post training static quantization technique and the original model itself by several seconds.

Since either quantization techniques did not yield a very efficient or a better model, we are proceeding with the fusion with MLC optimization and Puring with MLC optimization paths. The reason behind this may be the lack of calibration data to implement Static Quantization efficiently or also the lack of training dataset availability.

Pruning

In order to improve accuracy, speed and space efficiency, we carried out some experimentations with pruning, fusion and quantization techniques.

Pruning refers to the process of cutting down/removing some amount of information from the model which will result in a more compact, less computationally demanding better performing model. For our specific case, we worked with the following methods. Each of these pruning techniques were made to remove the pruned weights permanently.

- ***Unstructured pruning*** : For unstructured pruning, we resorted to L1 norm unstructured pruning which provided comparable results even with a high percentage of pruning. This type of pruning is carried out by computing the absolute difference between the 'weights' in this case. Unstructured pruning offered great results, even after setting a pruning percentage close to 0.45% still retaining an accuracy of 86%.
- ***Kernel pruning*** : Kernel pruning is a structured pruning technique where kernels are dropped based on the L2 norm metric. It provided the best accuracy compared to other

pruning methods but the ideal goal of model compression is somewhat underwhelming in this case. Kernel pruning provides close to original model results with low pruning percentages.

- **Filter pruning** : While kernel pruning performs decently on the model, filter pruning done based on a similar L2 norm set up performs very poorly. This might be due to the fact that only some information from the filter (maybe specific kernels) were not essential for the final task.

After experimenting with various pruning percentages, we found the **Unstructured L1 norm weight pruning** technique to be performing well in spite of pruning an impactful amount of weights.

MLC Optimizations

We did 9 MLC Optimizations namely :

1. `CombineParallelDense()`
2. `CombineParallelConv2D()`
3. `VectorizeLoop()`
4. `LoopPartition()`
5. `SimplifyInference()`
6. `Inline()`
7. `Eliminate Common Subexp()`
8. `FoldConstant()`
9. `AutoTVM Optimization`

In these optimization we can group based on what sort of optimization happens and at what level.

Operator Fusion Optimization : Operator Fusion is an optimization in deep learning compilers like TVM that combines multiple operators into one computation kernel. This reduces memory usage, minimizes kernel launch overhead, and improves execution speed through better data locality and parallelism, making models more efficient on CPUs, GPUs, and accelerators.

The operator fusion optimizations applied are :

1. `CombineParallelDense()` - Merges multiple parallel dense layers with the same input into one to reduce computation and memory overhead.
2. `CombineParallelConv2D()` - Merges multiple parallel conv2d layers with identical inputs to improve computation efficiency.

Inference Simplification Optimization : Inference Simplification is an optimization technique in deep learning compilers that simplifies a model's computation graph by removing redundant layers,

folding operations like BatchNorm into preceding layers, precomputing constants, and eliminating no-op layers. This reduces model size, memory usage, and inference latency, enabling faster and more efficient execution on various hardware backends.

The two inference simplification optimizations applied are :

1. *SimplifyInference()* - Simplifies models by folding operations such as BatchNorm into earlier layers like Conv2D, reducing runtime cost.
2. *Inline()* - Inlines functions to avoid function calls, enabling deeper optimization passes.

Loop Level Optimization : Loop-Level Optimization is a compiler technique that improves the efficiency of loops by transforming them to enhance parallel execution, reduce memory access, and minimize computation time. Common techniques include loop unrolling, tiling, fusion, vectorization, and partitioning. These optimizations help accelerate operations like matrix multiplication and convolution in deep learning models by improving data locality, parallelism, and execution speed.

The loop level optimizations applied are :

1. *VectorizeLoop()* - Converts loop computations into vectorized instructions, improving CPU and GPU utilization.
2. *LoopPartition()* - Splits loops into smaller independent sections, allowing more efficient scheduling and execution.

Expressions Simplification Optimization : Expression Simplification Optimization is a compiler technique that simplifies mathematical and computational expressions by reducing redundant or unnecessary calculations. It involves eliminating common subexpressions, folding constant values, simplifying algebraic operations, and removing dead code. This optimization reduces the computational workload, improves memory efficiency, and speeds up execution, making models more efficient during inference and training.

The expression simplification optimization applied are :

1. *EliminateCommonSubexp()* - Detects and removes common subexpressions to avoid redundant computation.
2. *FoldConstant()* - Simplifies the computation graph by precomputing constant expressions at compile time, reducing runtime overhead.

Auto Tuning :

Finally we did Autotuning which improved upon the custom relay model optimization and gave a better performance in terms of the running time of the algorithm. We used the XGB tuner to do the autotuning. We did 20 trials with the Autotuning to find the best model output.

Metrics Comparison - Original vs Optimized LPRNet

For the final optimized model, L1 norm unstructured pruning, fusion and dynamic quantization have been applied in order to retain / improve accuracy, time and memory consumption. The MLC optimization used after all the comparison trials are as follows.

<i>Metrics / Model</i>	<i>Original LPRnet model</i>	<i>Optimized LPRNet model</i>
<i>Test Accuracy</i>	<i>90.1%</i>	<i>89.4%</i>
<i>Inference Time</i>	<i>159.4 ms</i>	<i>152.25 ms</i>
<i>ONNX file size</i>	<i>1.71 mb</i>	<i>1.704 mb</i>

The final optimized model uses Autotuning for MLC optimization and Unstructured L1 norm pruning for model optimization in order to retain accuracy and compress resource requirements.

Lessons through the project

1. Sri Roopa Ramesh Babu :

- In terms of technical exposure, I have gained lots of insights on the order of executing model optimizations. I understood how each of the optimizations like fusion, quantization and pruning have a cascading effect in increasing the overall performance of the model.
- I also learnt about fusing various layers by considering their functionalities as operations during the Model Optimization phase.
- In terms of soft skills, I have gained useful experience in time management, task delegation and communication with peers for unified outcomes.

2. Saikrishna Rajaraman

- While trying to convert the pytorch model to the TVM TIR script, we faced a lot of issues. One of the main reasons being the MaxPool3D layer which needed a 5D input but we only had 4D input.
- The forward logic had extra processing which was difficult to implement using the tvn script.
- We were able to generate the tvn script with the backbone network.
- For custom MLC Optimizations it was difficult to come up with the right combination of the optimizations since it was a complex network of layers.

- Because of the MaxPool3D we were not able to do simplification optimization using the ONNX libraries

3. Chandana Mallu

- During the implementation of various quantization techniques to the model, I came across different quantization methods available to work with for the DNN model we had. Went through documentations to figure out which would be the most suitable technique.
- Encountered an error occurred because PyTorch's Post-Training Static Quantization (PTQ) currently does not support CUDA inference for some quantized operations, including Conv2d. PyTorch only supports static quantization on CPU backends like fbgemm or qnnpack.
- The CUDA backend in PyTorch lacks implementations for specific quantized operators like `quantized::conv2d.new`, which caused the `NotImplementedError`. I enabled the correct CPU backend and moved the model to the CPU.
- Dynamic Quantization was a better fit to our model than the PTQ.